Meeting 6B — Runtime Implementation Architecture, DMA Architecture, Interrupt Architecture, and System Execution Model

Phase: 3.1 — Runtime Architecture Design
Meeting Objective: Finalize the complete execution architecture of the Digital Harmonium Workstation. This document defines how the ESP32 operates internally during real-time execution and serves as the master runtime blueprint for future implementation.

---

1. Purpose

Previous meetings defined:

- System purpose
- Hardware
- User interface
- Runtime states
- Software modules
- Scheduling philosophy

This meeting answers the most fundamental engineering question:

> **How does the ESP32 actually execute all of these systems simultaneously without ever interrupting the audio?**

This document therefore defines the runtime operating model.

Nothing implemented later should violate the architecture described here.

---

2. Core Design Philosophy

The workstation is **not** organized around features.

It is organized around **time**.

Everything exists for one purpose:

> Produce continuous, uninterrupted audio every single audio block.

Everything else is secondary.

This means the runtime must behave deterministically.

Every execution cycle should always follow the same sequence.

Nothing random.

Nothing blocking.

Nothing unpredictable.

---

3. Runtime Layer Architecture

Rather than viewing the firmware as dozens of unrelated modules, the runtime is divided into four major execution layers.

Each layer has one responsibility.

Layer 1 — Hardware Timing Layer

Purpose

This layer exists solely to maintain perfect audio timing.

It never performs musical calculations.

It never processes voices.

It never updates displays.

It only maintains the hardware clock.

Responsibilities

- I2S peripheral
- DMA controller
- Audio clock generation
- DMA completion interrupt
- Buffer switching

Characteristics

- Highest priority in the entire firmware.
- Executes in microseconds.
- Never performs expensive operations.
- Must remain extremely small and deterministic.

Philosophy

The hardware layer never asks:

> "What sound should I generate?"

Instead it asks only:

> "Is the next audio buffer ready?"

Layer 2 — Audio Runtime Layer

Purpose

This is the computational heart of the workstation.

Every musical calculation happens here.

Every 512-sample audio block the Audio Runtime wakes, generates new audio, fills the next DMA buffer, then sleeps until the next request.

Responsibilities

- Read queued keyboard events
- Allocate voices
- Update active voices
- Advance envelopes
- Advance LFOs
- Process pitch
- Perform interpolation
- Apply filters
- Apply modulation
- Apply effects

- Mix voices
- Apply master gain
- Produce final PCM buffer

Characteristics

- Highest software priority.
- Executes once per audio block.
- Owns the majority of CPU time.
- Never performs filesystem operations.
- Never waits for external devices.

Philosophy

The Audio Runtime owns time.

Every other software system borrows whatever time remains.

Layer 3 — Control Runtime Layer

Purpose

Everything the musician interacts with belongs here.

Unlike the Audio Runtime, this layer never generates sound directly.

Instead it sends commands to the Audio Runtime.

Responsibilities

- Keyboard scanner
- Button scanner
- LCD updates
- LED updates
- Menu navigation
- Voice selection
- Tempo changes
- Recording controls
- Playback controls
- Settings changes

Characteristics

- Medium priority.
- Responsive but never time-critical.
- Completely separated from audio generation.

Philosophy

User actions become commands.

Commands become queued events.

Audio interprets those events safely during the next rendering cycle.

Layer 4 — Background Runtime Layer

Purpose

Everything slow belongs here.

If CPU time exists, these operations execute.

If CPU becomes busy, they simply wait.

Responsibilities

- Instrument loading
- Recording save
- SD card access
- USB communication
- Diagnostics
- Runtime statistics
- Configuration save
- Maintenance tasks

Characteristics

- Lowest priority.

- Never interrupts audio.
- May be paused indefinitely without affecting playback.

Philosophy

Background work never owns the processor.

It only occupies unused execution time.

---

4. Overall Runtime Flow

The workstation continuously repeats the following cycle.

```
DMA finishes current block
        │
        ▼
DMA Interrupt
        │
        ▼
Notify Audio Task
        │
        ▼
Audio Task wakes
        │
        ▼
Process queued commands
        │
        ▼
Update active voices
        │
        ▼
Generate 512 samples
        │
        ▼
Fill next DMA buffer
        │
        ▼
Sleep
```

This loop continues for the lifetime of the instrument.

---

5. DMA Architecture

DMA is responsible for transferring completed audio samples from memory to the I2S peripheral.

The CPU never manually outputs samples.

Instead, DMA performs all transfers automatically.

Double Buffer Strategy

Two independent audio buffers exist.

```
Buffer A

DMA → Playing

CPU → Idle
```

```
Buffer B

DMA → Idle

CPU → Rendering
```

After DMA finishes:

```
Swap

Buffer B becomes playback buffer

Buffer A becomes render buffer
```

Then the process repeats forever.

Advantages

- CPU and DMA never access the same buffer simultaneously.
- No synchronization conflicts.
- Continuous playback.

- Extremely deterministic.

---

6. DMA Interrupt Philosophy

The DMA interrupt must remain the smallest routine in the entire firmware.

It must never:

- Allocate voices
- Read keyboard
- Load files
- Generate samples
- Apply filters
- Update displays

Instead it performs only three operations.

DMA Interrupt Responsibilities

1. Detect buffer completion.
2. Swap active render/playback buffer.
3. Notify the Audio Task.

Immediately afterwards the interrupt exits.

Target Execution Time

Ideally

Less than

10 microseconds.

This keeps interrupt latency negligible.

---

7. Interrupt Architecture

Only a minimal number of interrupts should exist.

Interrupt 1 — DMA Complete

Purpose

Audio synchronization.

Responsibilities

- Buffer swap
- Wake Audio Task

Priority

Highest.

Interrupt 2 — GPIO (Optional)

Purpose

Wake processor from sleep or detect special external events.

Normal buttons continue to be scanned periodically by the keyboard task.

Interrupt 3 — Hardware Timer (Optional)

Reserved only if future features require precise periodic timing beyond the audio clock.

Examples

- Special metronome implementation
- Experimental timing features

Not required initially.

Philosophy

Fewer interrupts produce:

- Simpler firmware
- Easier debugging
- Greater predictability
- Lower interrupt latency

8. Internal Communication Architecture

Subsystems never directly invoke one another.

Instead standardized communication mechanisms are used.

Command Queue

Used for discrete events.

Examples

```
KEY_ON

KEY_OFF

VOICE_CHANGE

TEMPO_CHANGE

START_RECORDING

STOP_RECORDING
```

The Audio Runtime consumes these commands during the next rendering cycle.

Event Flags

Used for binary conditions.

Examples

```
Recording Active

USB Connected

Tanpura Running

Tabla Running
```

```
SD Mounted
```

No complex data transfer required.

Shared Runtime Variables

Used for persistent system settings.

Examples

- Master volume
- Current transpose
- Selected instrument
- Current tempo

These variables are updated infrequently and read by multiple modules.

---

9. Canonical Runtime Data Flow

All runtime communication follows one direction.

```
Keyboard

Buttons

Recorder

USB

↓

Control Runtime

↓

Command Queue

↓

Audio Runtime
```

```
↓

Voice Manager

↓

Voice Engine

↓

Mixer

↓

Master Output

↓

DMA Buffer

↓

I2S Peripheral

↓

MAX98357

↓

Speaker / Headphones
```

No circular dependencies should exist.

---

10. CPU Ownership

The processor should conceptually be divided as follows.

Runtime Layer	Approximate CPU Ownership
Audio Runtime	~70%
Control Runtime	~15%
Display Runtime	~5%

Runtime Layer	Approximate CPU Ownership
Background Runtime	~10%

These percentages are design goals rather than fixed measurements.

The important principle is that audio always dominates CPU usage.

---

11. Safety Margin

Even when everything functions correctly, the CPU should never operate continuously at maximum capacity.

Approximately two milliseconds of execution time should remain unused each audio block.

This reserve absorbs temporary workload spikes caused by:

- Multiple simultaneous key presses
- Tabla transitions
- Recording events
- Display updates
- Additional voice allocation

Without this reserve, occasional overloads would immediately produce audible glitches.

---

12. Runtime Design Rules

The implementation must obey the following rules.

1. Audio rendering always has highest priority.
2. DMA interrupts remain extremely short.
3. No blocking operations inside the audio path.
4. Filesystem operations execute only in background tasks.
5. Every runtime layer has one clearly defined responsibility.
6. Every communication path follows queues, flags, or shared runtime variables.
7. CPU execution must remain deterministic.
8. Audio timing must never depend on UI activity.
9. Every audio block follows exactly the same execution sequence.
10. Simplicity is preferred over unnecessary optimization.

---

13. Final Runtime Philosophy

The Digital Harmonium Workstation should not behave like a collection of independent features.

Instead it should behave like a professional embedded musical instrument whose entire runtime is synchronized around one immutable heartbeat:

> **Every 11.6 milliseconds, produce the next block of perfect audio.**

Everything else—menus, displays, recording, settings, storage, diagnostics, and future expansion—must be designed to support that single objective without ever compromising it.

This runtime architecture forms the permanent foundation upon which all subsequent Phase 3 implementation work will be built.